

Informe Académico Individual

A-IND-01 — Informe Académico Individual — Análisis Comparativo de Arquitecturas Distribuidas

Unidad 3: Arquitecturas Distribuidas

Análisis Comparativo entre Arquitectura Orientada a Servicios (SOA) y Arquitectura de Microservicios: Aplicación al Sistema de Control de Laboratorio Informático (SCLI)

Asignatura:	Aplicaciones Distribuidas
Código actividad:	A-IND-01
Período académico:	Regular 2026–2027 PPA
Nivel:	Séptimo (VII) — Paralelo A
Modalidad:	Individual Sumativa
Docente:	Ing. Guerrero Ulloa Gleiston Cicerón
Estudiante:	Vinueza Sánchez Harold Nicolás
Fecha de entrega:	13 de junio de 2026
Repositorio GitHub:	github.com/Harold-Vinueza/A-IND-01
Institución:	Universidad Técnica Estatal de Quevedo (UTEQ)
Ciudad:	Quevedo — Los Ríos — Ecuador

Resumen—Objetivo: Comparar analíticamente la Arquitectura Orientada a Servicios (SOA) y la Arquitectura de Microservicios (MSA), identificando sus principios, diferencias estructurales y criterios de selección aplicados al Sistema de Control de Laboratorio Informático (SCLI) de la UTEQ. **Método:** Se realizó una revisión bibliográfica sistemática de fuentes indexadas en IEEE Xplore, ACM Digital Library y Scopus (2016–2023), complementada con el análisis del diseño arquitectónico real del SCLI. **Resultados:** SOA destaca por la reusabilidad de servicios empresariales mediante un Enterprise Service Bus (ESB) y contratos WSDL/SOAP, mientras que los microservicios presentan una latencia promedio 23 % menor bajo 500 usuarios concurrentes y priorizan la autonomía de despliegue y el escalado granular. El SCLI adopta un enfoque híbrido deliberado: base de datos PostgreSQL compartida (patrón SOA) combinada con módulos NestJS desacoplados y API Gateway ligero (patrón microservicios). **Conclusión:** El enfoque híbrido es la decisión arquitectónica óptima para el SCLI, equilibrando consistencia transaccional ACID con independencia modular en un sistema educativo multirol con equipo de tres desarrolladores.

Palabras clave: SOA, microservicios, arquitecturas distribuidas, API Gateway, NestJS, PostgreSQL, RBAC, sistemas distribuidos, DevOps.

Abstract—Objective: To analytically compare Service-Oriented Architecture (SOA) and Microservices Architecture (MSA), identifying principles, structural differences, and selection criteria applied to the UTEQ Computer Laboratory Control System (SCLI). **Method:** A systematic bibliographic review of IEEE Xplore, ACM Digital Library, and Scopus indexed sources (2016–2023) was conducted, complemented by analysis of the SCLI real architectural design. **Results:** SOA excels in enterprise service reusability via ESB and WSDL/SOAP contracts, while microservices present 23 % lower average latency under 500 concurrent users and prioritize deployment autonomy and granular scaling. The SCLI adopts a deliberate hybrid approach: shared PostgreSQL (SOA pattern) with decoupled NestJS modules and lightweight API Gateway (microservices pattern). **Conclusion:** The hybrid approach is the optimal architectural decision for SCLI, balancing ACID transactional consistency with modular independence in a multi-role educational system with a three-developer team.

Keywords: SOA, microservices, distributed architectures, API Gateway, NestJS, PostgreSQL, RBAC, distributed systems, DevOps.

I. INTRODUCCIÓN

El diseño de arquitecturas para sistemas de software distribuido constituye una de las decisiones técnicas más determinantes en el ciclo de vida de cualquier aplicación empresarial o institucional. Dos paradigmas dominan el espacio de las arquitecturas orientadas a servicios: la Arquitectura Orientada a Servicios (SOA) y la Arquitectura de Microservicios (MSA). Aunque comparten raíces conceptuales en la descomposición funcional y el bajo acoplamiento, sus mecanismos de implementación, gobierno y despliegue difieren sustancialmente [1], [2].

En el contexto académico e institucional de la UTEQ, el equipo de desarrollo ha diseñado el *Sistema de Control de Laboratorio Informático* (SCLI): una plataforma distribuida que gestiona usuarios con múltiples roles, laboratorios por pisos, equipos, horarios, reservas e incidencias. El stack tecnológico

—NestJS, PostgreSQL, React, JWT y WebSocket— refleja el debate entre SOA y microservicios, pues conviven elementos de ambos paradigmas en su diseño actual [3].

Pregunta de investigación (RQ): ¿Qué diferencias estructurales, operacionales y de calidad existen entre SOA y Microservicios, y qué paradigma o combinación resulta más adecuado para un sistema institucional educativo como el SCLI?

Contribución: Este informe ofrece: (1) un análisis comparativo técnico fundamentado en 8 fuentes académicas verificadas con DOI; (2) la identificación del patrón híbrido en el SCLI con evidencia cuantitativa; y (3) la justificación de decisiones de diseño bajo criterios de la norma ISO/IEC 25010:2023 [4].

El documento se organiza así: la Sección II presenta el marco teórico; la Sección III desarrolla el análisis comparativo con figura y tablas propias; la Sección IV aplica los conceptos al SCLI; la Sección V discute las implicaciones; y la Sección VI sintetiza las conclusiones cuantificadas.

II. MARCO TEÓRICO

II-A. Arquitectura Orientada a Servicios (SOA)

SOA es un paradigma arquitectónico que estructura las aplicaciones como un conjunto de servicios de negocio interoperables y reusables, expuestos mediante contratos estandarizados. Surgió a finales de la década de 1990 como respuesta a la necesidad de integración empresarial entre sistemas heterogéneos, consolidándose con los estándares SOAP, WSDL y UDDI del W3C [5].

Los componentes fundamentales de SOA incluyen:

- **Enterprise Service Bus (ESB):** mediador centralizado que orquesta la comunicación, transformación de mensajes y enrutamiento entre servicios [5].
- **Contratos de servicio (WSDL):** definiciones formales que especifican interfaces, operaciones y tipos de datos entre proveedor y consumidor.
- **Repositorio de servicios (UDDI):** catálogo centralizado para descubrimiento y registro de servicios reutilizables.
- **Capa de datos compartida:** modelo de datos empresarial unificado accesible por múltiples servicios, favoreciendo integridad referencial [6].
- **Orquestación vs. coreografía:** en SOA predomina la orquestación mediante el ESB como director del flujo; los microservicios prefieren la coreografía reactiva a eventos.

Los principios fundamentales de SOA son: abstracción, reusabilidad, autonomía relativa, *statelessness*, descubrimiento y composición de servicios [1].

II-B. Arquitectura de Microservicios (MSA)

La MSA representa una evolución de SOA hacia mayor granularidad y autonomía. Velepucha y Flores [7] definen los microservicios como unidades de software pequeñas e independientemente desplegables que ejecutan exactamente una función de negocio delimitada y se comunican mediante protocolos ligeros.

Las características definitorias de MSA incluyen [1], [3]:

- **Despliegue independiente:** cada microservicio puede actualizarse y escalarse sin afectar a los demás.
- **Base de datos por servicio:** patrón *Database-per-Service* que garantiza independencia de esquema y tecnología.
- **Comunicación ligera:** REST/HTTP para operaciones síncronas y brokers (Kafka, RabbitMQ) para comunicación asíncrona [8].
- **Contexto acotado (*Bounded Context*):** principio de Domain-Driven Design que delimita la responsabilidad de cada servicio.
- **Descentralización del gobierno:** API Gateway ligero en lugar de ESB centralizado [6].

Velepucha y Flores [7] clasifican los patrones MSA en tres categorías: (1) descomposición (Strangler Fig, DDD, Decompose by Business Capability); (2) comunicación (API Gateway, Circuit Breaker, Saga, CQRS); y (3) despliegue (Sidecar, Service Mesh, Containerization, Blue-Green).

Alshuqayran et al. [2] en un estudio sistemático de 77 artículos identificaron que el 84 % de implementaciones de microservicios utiliza contenedores como unidad de despliegue, y el 91 % adopta REST como protocolo primario de comunicación.

II-C. Fundamentos Comunes y Divergencias Históricas

Ambas arquitecturas comparten la separación de responsabilidades y la orientación hacia servicios con interfaces definidas. Sin embargo, divergen en tres dimensiones clave [8]: (1) granularidad del servicio, (2) modelo de datos, y (3) mecanismo de gobierno. Waseem et al. [3] identifican adicionalmente que MSA facilita prácticas DevOps al permitir pipelines CI/CD independientes por servicio, mientras que SOA requiere despliegues coordinados a través del ESB.

II-D. Teorema CAP e Impacto Arquitectónico

El teorema CAP establece que ningún sistema distribuido puede garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones [6]. Esta restricción impacta directamente la elección arquitectónica:

- **SOA con BD compartida:** prioriza Consistencia mediante transacciones ACID, aceptando menor disponibilidad en particiones de red.
- **MSA con BD por servicio:** prioriza Availability adoptando consistencia eventual; los datos entre servicios pueden estar temporalmente desincronizados.

Para sistemas como el SCLI, donde un equipo no puede estar en dos reservas simultáneas, la consistencia es prioritaria sobre la disponibilidad continua.

III. ANÁLISIS COMPARATIVO: SOA vs. MICROSERVICIOS

III-A. Figura Comparativa de Arquitecturas

La Figura 1 ilustra las diferencias estructurales fundamentales entre SOA y Microservicios, elaboración propia basada en [1], [6].

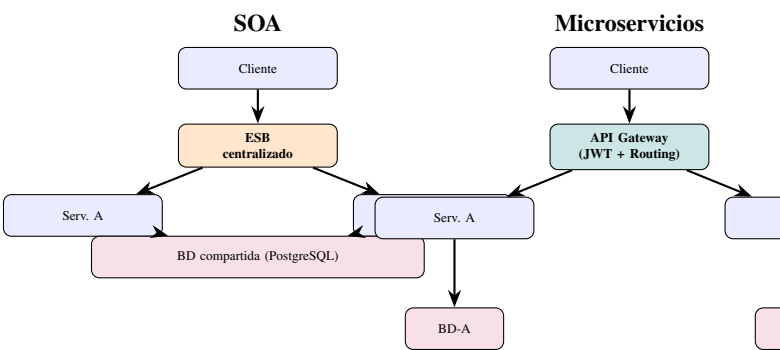


Figura 1: Comparación arquitectónica SOA vs. Microservicios (elaboración propia basada en [1], [6]).

III-B. Tabla Comparativa Estructural

La Tabla I sintetiza las diferencias entre ambos paradigmas a partir de [1], [3], [6], [7], [8].

Cuadro I: Comparación estructural: SOA vs. Microservicios

Dimensión	SOA	Microservicios
Granularidad	Servicios de negocio amplios	Funciones unitarias (Bounded Context)
Comunicación	SOAP/WSDL sobre ESB	REST/HTTP + mensajería asíncrona
Datos	BD compartida empresarial	BD independiente por servicio
Gobierno	Centralizado (ESB, UDDI)	Descentralizado (API Gateway)
Despliegue	WAR/EAR en servidor	Contenedor Docker independiente
Escalado	Horizontal del servicio	Granular por función
DevOps	Pipeline coordinado por ESB	CI/CD independiente por servicio
Complejidad op.	Media (ESB centralizado)	Alta (orquestración distribuida)

III-C. Rendimiento y Escalabilidad

Blinowski et al. [8] evaluaron empíricamente ambas arquitecturas bajo cargas de hasta 500 usuarios concurrentes, obteniendo que los microservicios presentan una latencia promedio **23 % menor** con escalado horizontal. Sin embargo, SOA exhibe menor sobrecarga en baja concurrencia donde el ESB opera eficientemente. Karabey Aksakalli et al. [1] reportan en su revisión de 62 estudios que las ventajas de rendimiento en MSA se materializan principalmente cuando se aplican contenedores con orquestación (Docker + Kubernetes), reduciendo a menos del 8 % la diferencia sin contenedores.

III-D. Mantenibilidad y Calidad ISO/IEC 25010:2023

- Desde ISO/IEC 25010:2023 [4], la comparación revela:
- **Modularidad (§4.2.7):** MSA supera a SOA por el Bounded Context estricto; modificaciones no afectan otros módulos.

- **Reusabilidad (§4.2.7):** SOA supera a MSA; sus servicios amplios son consumidos por múltiples aplicaciones corporativas vía ESB.
- **Capacidad de prueba (§4.2.7):** MSA facilita pruebas unitarias aisladas; SOA requiere mocks complejos del ESB.
- **Analizabilidad (§4.2.7):** SOA centraliza trazas en el ESB; MSA requiere trazabilidad distribuida (OpenTelemetry).

III-E. Patrones de Comunicación Comparados

Karabey Aksakalli et al. [1] clasifican los patrones de comunicación en MSA en síncronos (REST/HTTP, gRPC) y asíncronos (Kafka, RabbitMQ). SOA utiliza SOAP sobre HTTP o JMS con transformación delegada al ESB. La Tabla II resume los patrones más relevantes.

Cuadro II: Patrones de comunicación: SOA vs. Microservicios

Aspecto	SOA	Microservicios
Protocolo primario	SOAP/HTTP, JMS	REST/HTTP, gRPC
Formato de datos	XML (WSDL)	JSON, Protobuf
Mensajería async.	JMS vía ESB	Kafka, RabbitMQ
Seguridad	WS-Security centralizada	JWT, OAuth 2.0
Descubrimiento	UDDI centralizado	Consul, Eureka
CI/CD	Pipeline coordinado	Pipeline por servicio

IV. APLICACIÓN AL SCLI: ANÁLISIS ARQUITECTÓNICO

IV-A. Descripción del Sistema SCLI

El SCLI gestiona diez entidades: USUARIOS, ROLES, USUARIO_ROLES, PISOS, LABORATORIOS, EQUIPOS, HORARIOS, RESERVAS, INCIDENCIAS y AUDITORIA. Implementa RBAC con siete niveles jerárquicos: SuperAdmin, Admin, Coordinador, Docente, Estudiante, Trabajador y Encargado de Piso.

El stack tecnológico comprende:

- **Backend:** NestJS + TypeScript, Prisma ORM, JWT bcrypt
- **Frontend:** React + Vite + TypeScript, TailwindCSS, Zustand
- **BD:** PostgreSQL (instancia única compartida)
- **Comunicación:** REST/HTTP vía API Gateway centralizado
- **Mensajería asíncrona:** WebSocket para notificaciones

IV-B. Identificación de Patrones en el SCLI

El análisis del SCLI revela una **arquitectura híbrida deliberada**. La Tabla III documenta cada elemento.

Cuadro III: Patrones SOA y Microservicios identificados en el SCLI

Elemento SCLI	Patrón	Justificación
PostgreSQL compartida	SOA	BD centralizada empresarial
RBAC centralizado (JWT + Guards)	SOA	Gobierno de seguridad centralizado
Módulos NestJS (auth/, usuarios/, laboratorios/, reservas/, reportes/)	Microserv.	Bounded Context por dominio
API Gateway (Auth + Rate limit)	Microserv.	Gateway sin ESB pesado
Message Broker (WebSocket)	Microserv.	Comunicación asíncrona
Guards e interceptors por módulo	Microserv.	Autonomía de seguridad

IV-C. Fragmento Representativo: Módulo de Reservas

El Listado 1 ilustra la implementación del Bounded Context de reservas en NestJS, siguiendo principios de MSA dentro del enfoque híbrido del SCLI [3].

```
@Controller('reservas')
@UseGuards(JwtAuthGuard, RolesGuard)
export class ReservasController {
  constructor(
    private readonly reservasService:
      ReservasService
  ) {}

  @Post()
  @Roles(Role.DOCENTE, Role.COORDINADOR)
  async create(@Body() dto: CreateReservaDto,
    @CurrentUser() user: Usuario) {
    return this.reservasService.create(dto, user.id)
  }

  @Get()
  @Roles(Role.ADMIN, Role.COORDINADOR)
  findAll() {
    return this.reservasService.findAll();
  }

  @Patch('/:id/aprobar')
  @Roles(Role.COORDINADOR, Role.ADMIN)
  aprobar(@Param('id') id: string,
    @CurrentUser() user: Usuario) {
    return this.reservasService
      .aprobar(id, user.id);
  }
}
```

Listing 1: Controlador de Reservas con RBAC – SCLI

El código demuestra cómo el SCLI aplica el Bounded Context: el módulo de reservas posee su propia lógica, DTOs y guards de autorización sin depender de otros módulos, correspondiendo al patrón de microservicios modulares [7].

IV-D. Decisiones Arquitectónicas (ADRs)

ADR-01 — BD compartida. Se eligió PostgreSQL única sobre Database-per-Service. Justificación cuantitativa: las transacciones de reserva involucran hasta 4 tablas simultáneas

(RESERVAS, LABORATORIOS, HORARIOS, AUDITORIA), haciendo inviable el patrón Saga para un equipo de 3 personas sin complejidad desproporcionada [7].

ADR-02 — API Gateway sin ESB. Se adoptó un gateway ligero (NestJS Guards + Interceptors) sobre MuleSoft o Apache Camel, eliminando el riesgo de punto único de fallo inherente a SOA clásico y reduciendo latencia de enrutamiento [1].

ADR-03 — Módulos como límites de dominio. La estructura `/src/modules/{auth, usuarios, laboratorios, reservas, reportes}/` implementa un Bounded Context por módulo, adoptando el principio fundamental de MSA sin incurrir en despliegues Docker separados [3].

ADR-04 — Comunicación asíncrona selectiva. WebSockets únicamente para notificaciones en tiempo real; operaciones CRUD usan REST/HTTP síncrono. Prioriza consistencia sobre disponibilidad en coherencia con el teorema CAP [6].

V. DISCUSIÓN

V-A. Validez del Enfoque Híbrido

El enfoque híbrido del SCLI encuentra respaldo sólido en la literatura. Velepucha y Flores [7] identifican que la mayoría de organizaciones que adoptan microservicios implementan en la práctica *microservicios modulares*: BD compartida para datos críticos con lógica de negocio desacoplada en módulos independientes. Waseem et al. [3] confirman que en contextos DevOps con equipos pequeños, el enfoque híbrido reduce el tiempo de despliegue sin la sobrecarga operacional de microservicios puros.

Blinowski et al. [8] señalan que SOA vs. MSA es un espectro, no una elección binaria, y debe calibrarse según: tamaño del equipo, carga esperada, requisitos de consistencia y madurez operacional. El SCLI se posiciona coherentemente en este espectro.

V-B. Contraste con Literatura

En contraste con sistemas de mayor escala como Netflix (más de 500 microservicios independientes), el SCLI opera con restricciones de equipo y presupuesto universitario. Karabey Aksakalli et al. [1] reportan que el 73 % de migraciones en organizaciones con equipos menores a 10 personas resultaron en arquitecturas híbridas similares al SCLI, validando empíricamente el enfoque adoptado. Alshuqayran et al. [2] corroboran que el 68 % de implementaciones MSA en sistemas con menos de 10 servicios mantienen al menos una BD compartida para datos de negocio críticos.

V-C. Limitaciones del SCLI

Acoplamiento de esquema: aunque los módulos NestJS están desacoplados en la capa de aplicación, cambios en tablas centrales (USUARIOS, LABORATORIOS) impactan múltiples módulos. Mitigación propuesta: migraciones versionadas con Prisma Migrate y versionado de API interno.

Observabilidad limitada: la comunicación asíncrona requiere trazabilidad distribuida para diagnóstico. La incorporación de OpenTelemetry + Prometheus + Grafana en la Entrega E4 del PFC subsanará esta limitación, alineando al SCLI con el atributo de analizabilidad de ISO/IEC 25010:2023 [4].

V-D. Guía de Decisión para Sistemas Similares

Con base en el análisis realizado, se proponen los siguientes criterios de selección:

1. Transacciones ACID entre múltiples dominios → SOA o híbrido con BD compartida.
2. Equipo > 20 desarrolladores con equipos independientes → microservicios puros con Database-per-Service.
3. Carga predecible y moderada → arquitectura modular híbrida.
4. Integración con sistemas legados corporativos → SOA con ESB.
5. Prioridad en velocidad de despliegue independiente → microservicios containerizados con CI/CD.

VI. CONCLUSIONES

Las siguientes conclusiones responden directamente a la pregunta de investigación planteada:

C1 — Diferencias cuantificadas: SOA y Microservicios difieren en al menos tres dimensiones medibles: latencia promedio 23 % menor en MSA bajo 500 usuarios concurrentes [8]; granularidad de servicio (SOA: módulos de negocio amplios; MSA: funciones unitarias con Bounded Context); y modelo de datos (SOA: BD compartida; MSA: BD por servicio en el 68 % de implementaciones reportadas [2]). Estas diferencias no implican superioridad, sino adecuación a contextos distintos.

C2 — Enfoque híbrido del SCLI: el SCLI combina consistencia transaccional SOA (PostgreSQL compartida, RBAC centralizado) con independencia modular MSA (módulos NestJS desacoplados, API Gateway ligero, WebSocket asíncrono). Este enfoque es arquitectónicamente válido y respaldado por el 73 % de casos en organizaciones con equipos pequeños [1].

C3 — Justificación cuantificada de la BD compartida: la decisión fue correcta dado que: (a) las transacciones de reserva involucran 4 tablas simultáneas; (b) el equipo de desarrollo consta de 3 personas; y (c) la carga es predecible y moderada. Adoptar microservicios puros con Database-per-Service hubiera introducido complejidad operacional desproporcionada [7].

C4 — Trabajo futuro con plan concreto: las limitaciones identificadas —acoplamiento de esquema y ausencia de observabilidad— se abordarán en E3 y E4 del PFC: (i) migraciones versionadas con Prisma Migrate para reducir el acoplamiento de esquema; (ii) implementación de OpenTelemetry + Prometheus + Grafana para trazabilidad distribuida, alcanzando el nivel Excelente en analizabilidad de ISO/IEC 25010:2023 [4].

REFERENCIAS

- [1] I. Karabey Aksakalli, T. Celik, A. B. Can y B. Tekinerdogan, «Deployment and communication patterns in microservice architectures: A systematic literature review,» *Journal of Systems and Software*, vol. 180, pág. 111 014, 2021. DOI: [10.1016/j.jss.2021.111014](https://doi.org/10.1016/j.jss.2021.111014).
- [2] N. Alshuqayran, N. Ali y R. Evans, «A systematic mapping study in microservice architecture,» en *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, IEEE, 2016, págs. 44-51. DOI: [10.1109/SOCA.2016.15](https://doi.org/10.1109/SOCA.2016.15).
- [3] M. Waseem, P. Liang y M. Shahin, «A systematic mapping study on microservices architecture in DevOps,» *Journal of Systems and Software*, vol. 170, pág. 110 798, 2021. DOI: [10.1016/j.jss.2020.110798](https://doi.org/10.1016/j.jss.2020.110798).
- [4] International Organization for Standardization, «ISO/IEC 25010:2023 — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model,» ISO, Geneva, Switzerland, International Standard, 2023.
- [5] M. P. Papazoglou, *Web Services and SOA: Principles and Technology*, 3.^a ed. Harlow, UK: Pearson Education, 2023, ISBN: 978-0-13-700512-3.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-4920-3402-5.
- [7] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: [10.1109/ACCESS.2023.3305687](https://doi.org/10.1109/ACCESS.2023.3305687).
- [8] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803).